

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

RL-GAN: Adaptive Image Inpainting using Reinforcement Learning Guided GAN

A PPO Contextual-Bandit Controller for Strategy Selection on a Frozen DeepFill-v2 Generator

By

Hariprasaadh K (23BAI1061)
Nithish Kumar CS (23BAI1035)

A project report submitted to

Dr. Bharadwaja Kumar G

Professor, School of Computer Science and Engineering

in partial fulfilment of the requirements for the course

BCSE432E – Reinforcement Learning

in

B. TECH., COMPUTER SCIENCE AND ENGINEERING

Vellore Institute of Technology, Chennai

Vandalur – Kelambakkam Road, Chennai – 600 127

OCTOBER 2026

ABSTRACT

Deep image inpainting networks usually apply one fixed restoration strategy to every damaged region, even though a thin scratch, a seam along an object boundary, and a large textured hole demand different treatment. This project asks whether a reinforcement-learning controller can read a short description of the damage and pick a restoration strategy per image, and whether such control becomes more valuable as the missing area grows.

We built **RL-GAN**, an end-to-end pipeline on top of a frozen, Places2-pretrained **DeepFill-v2** generator. Four small, FiLM-conditioned *strategy adapters* (Global, Local, Boundary and Texture) are pretrained with four different losses on COCO val2017 images with random stroke masks. A Stable-Baselines3 **PPO** agent, framed as a one-step contextual bandit ($\gamma = 0$), observes a 261-dimensional state (a 256-d embedding of the backbone bottleneck, four mask-geometry statistics and one boundary-gradient proxy) and selects one adapter. The reward combines normalised PSNR, SSIM and LPIPS gains, a discriminator term and a per-action cost. The system ships with a command-line pipeline, a four-stage ablation harness and an interactive web studio.

Evaluation follows four stages: backbone only (A), a fixed Global adapter (B), the PPO-selected adapter in a single forward pass (C), and, labelled as an oracle, the PPO choice plus 400 test-time optimisation steps that use ground truth (D). On three test scenes the fixed Global adapter improves on the backbone in two of three scenes (+0.76 dB and +0.82 dB); the PPO-selected adapter matches it on the meadow scene (+0.82 dB over the backbone) but chooses the Boundary adapter on the other two scenes and falls 2.21 dB and 3.78 dB below the backbone. The oracle reaches 37.2–43.2 dB, showing the capacity of the adapter structure.

The report closes with future work: richer state encoders and rewards, longer policy training, stronger baselines, and evaluation across the four severity buckets.

Keywords: *image inpainting, reinforcement learning, PPO, contextual bandit, DeepFill-v2, FiLM, strategy selection.*

TABLE OF CONTENTS

1. INTRODUCTION 1
1.1 Background and motivation 1
1.2 Problem statement 1
1.3 Objectives 1
1.4 Contributions 2
1.5 Organisation of the report 2
2. BACKGROUND AND RELATED WORK 3
2.1 Deep generative image inpainting 3
2.2 Feature-wise Linear Modulation (FiLM) 3
2.3 Reinforcement learning, bandits and PPO 3
2.4 Perceptual and structural quality metrics 3
2.5 Positioning of this work 4
3. SYSTEM DESIGN AND METHODOLOGY 5
3.1 Overview 5
3.2 Frozen DeepFill-v2 backbone 5
3.3 State representation 5
3.4 Action space and strategy adapters 6
3.5 Adapter pretraining 7
3.6 Reward design 7
3.7 PPO contextual-bandit environment 8
3.8 Data, masks and severity buckets 8
3.9 Evaluation protocol 9
4. IMPLEMENTATION 10
4.1 Software stack 10
4.2 Code organisation 10
4.3 Pipeline workflow 10
4.4 Interactive web studio 11
5. RESULTS AND ANALYSIS 13
5.1 Experimental setup 13
5.2 Backbone-only baseline (Experiment A) 13
5.3 Do the four adapters behave differently? 13
5.4 Four-stage ablation 14
5.5 Web-studio behaviour and latency 17
5.6 Discussion 17
6. CONCLUSION AND FUTURE WORK 18
6.1 Conclusion 18
6.2 Future work 18
REFERENCES 19
APPENDIX A: REPRODUCTION COMMANDS 21
APPENDIX B: ADDITIONAL QUALITATIVE RESULTS 21

1. INTRODUCTION

1.1 Background and motivation

Image inpainting is the task of filling missing or corrupted regions of an image with content that is visually plausible and consistent with the surrounding pixels. It underpins photo restoration, object removal, and editing tools. Modern generative inpainting networks such as DeepFill-v2 combine gated convolutions with contextual attention to synthesise structure and texture from distant image regions, and a single trained generator is normally applied to every image in exactly the same way.

In practice, the defects that users create differ greatly. A thin scratch is mostly a boundary problem: the filled pixels must continue edges that cross the hole. A large hole in a grassy field is a semantic and texture problem. A hole that touches a sharp horizon or a reflective water surface is a seam problem. A single compromise network tends to blur edges in some cases and over-smooth texture in others. This motivates a controller that chooses *how* to restore, not only *what* to generate.

Reinforcement learning (RL) is a natural framework for such a decision: the controller observes a state, takes an action, and receives a scalar reward based on measured quality. Because each image needs one decision and the outcome is evaluated immediately, the problem is a *contextual bandit*, the one-step special case of a Markov decision process (MDP).

1.2 Problem statement

Given a damaged image I and a binary hole mask M, select one of $K = 4$ restoration strategies so that the completed image maximises a reward that combines reconstruction fidelity, perceptual similarity and computational cost. The central research question of this work is:

> *Can a PPO controller learn to choose among Global, Local, Boundary and Texture restoration strategies from the hole geometry and contextual features, and does adaptive control help more as the missing region becomes harder to reconstruct?*

1.3 Objectives

- Build a reproducible inpainting pipeline around a frozen pretrained generator (DeepFill-v2) so that all gains can be attributed to the adapters and the controller.
- Design four lightweight strategy adapters, each trained with a different objective, and verify that they behave differently.
- Formulate strategy selection as a one-step contextual bandit with a low-cost state vector, and train it with PPO.
- Evaluate honestly with a four-stage ablation that separates feed-forward inference from test-time optimisation using ground truth.
- Provide a usable web interface to demonstrate the system on user-drawn masks.

1.4 Contributions

- **A zero-extra-cost state.** The observation re-uses the backbone's own coarse-stage bottleneck instead of running a separate vision network, and adds only five scalar descriptors.
- **Specialised adapters.** Four FiLM-conditioned residual adapters pretrained with Global, Local, Boundary and Texture objectives.
- **A strict evaluation protocol.** Stages A–D keep valid baselines (A, B, C) apart from the ground-truth-supervised oracle (D).

1.5 Organisation of the report

Chapter 2 reviews the background. Chapter 3 describes the method: state, actions, adapters, reward and training. Chapter 4 explains the implementation and the web studio. Chapter 5 presents the experiments and analysis, and Chapter 6 presents the conclusion and future work.

2. BACKGROUND AND RELATED WORK

2.1 Deep generative image inpainting

Early deep inpainting methods used a single feed-forward encoder–decoder trained with reconstruction and adversarial losses. Yu *et al.* introduced **contextual attention**, a two-stage coarse-to-fine design in which the second stage copies and adapts features from known background patches [1]. **DeepFill-v2** replaced ordinary convolutions with **gated convolutions** that learn a soft, per-pixel and per-channel validity mask, which is well suited to free-form holes [2]:

$$O = \varphi(W_f * X) \odot \sigma(W_g * X) \tag{1}$$

where W_f and W_g are the feature and gating kernels, φ is an ELU activation and σ the sigmoid. DeepFill-v2 also uses a spectral-normalised PatchGAN discriminator [3], [4]. More recent systems such as LaMa use Fourier convolutions for large-mask inpainting [5], and diffusion models give high perceptual quality at a much higher inference cost. We choose DeepFill-v2 because pretrained Places2 weights are available [6], it runs in a single pass, and its two-stage structure exposes a natural point (the refine-stage output) at which adapters can be attached.

2.2 Feature-wise Linear Modulation (FiLM)

FiLM conditions a network by applying a learned, input-dependent affine transform to intermediate features [7]. Given a conditioning vector e, a small MLP predicts per-channel scale $\gamma(e)$ and shift $\beta(e)$:

$$\mathrm{FiLM}(h \mid e) = (1 + \gamma(e)) \odot h + \beta(e) \tag{2}$$

In this work e is a learned 64-dimensional embedding of the chosen strategy, so the same adapter structure can be specialised by its embedding and its training loss.

2.3 Reinforcement learning, bandits and PPO

An MDP is defined by states, actions, transition dynamics, a reward and a discount factor γ [8]. A **contextual bandit** is the special case in which an episode has exactly one step, so no future states exist and $\gamma = 0$ [9]. The expected return of an action is then just the immediate reward, and the advantage estimate reduces to $A = R - V(s)$. **Proximal Policy Optimisation (PPO)** is a policy-gradient method that constrains each update with a clipped surrogate objective [10]:

$$L^{CLIP}(\theta) = E_t[\, \min(\, r_t(\theta)\, A_t\,,\, \mathrm{clip}(\, r_t(\theta),\, 1-\varepsilon,\, 1+\varepsilon\,)\, A_t\,)\,] \tag{3}$$

with probability ratio $r_t(\theta) = \pi_\theta(a_t \mid s_t) / \pi_{old}(a_t \mid s_t)$ between the new and old policies. PPO is robust, easy to tune and available in Stable-Baselines3 [11], which we use together with the Gymnasium environment API [12].

2.4 Perceptual and structural quality metrics

Peak signal-to-noise ratio (PSNR) measures pixel-level fidelity, the structural similarity index (SSIM) compares local luminance, contrast and structure [13], and LPIPS compares deep VGG

features and correlates better with human perception [14]. PSNR rewards smooth averages and can penalise sharper but plausible textures, so we use all three in the reward and in the evaluation.

2.5 Positioning of this work

Prior inpainting work focuses on better single generators. We instead keep the generator fixed and study the *decision layer* above it: a small, cheap controller that selects among specialised refinements. This follows the general idea of using RL to choose among actions whose effect is measured by an image-quality reward, applied to a modern gated-convolution inpainting backbone.

3. SYSTEM DESIGN AND METHODOLOGY

3.1 Overview

Figure 1 shows the full pipeline. A damaged image and its hole mask are processed by the frozen DeepFill-v2 coarse stage. The coarse composite and the coarse-stage bottleneck feed a state builder that produces a 261-dimensional observation. The PPO policy selects an action $a \in \{0, 1, 2, 3\}$; the frozen refine stage produces an RGB refinement, and the selected adapter post-processes it. Only the hole region of the adapter output is used:

$$\hat{I} = (1 - M) \odot I + M \odot A_a(\, G_{refine}(I_c, M), e_a, M\,) \quad (4)$$

where I_c is the coarse composite, G_{refine} the frozen refine stage and A_a the selected adapter conditioned on its embedding e_a.

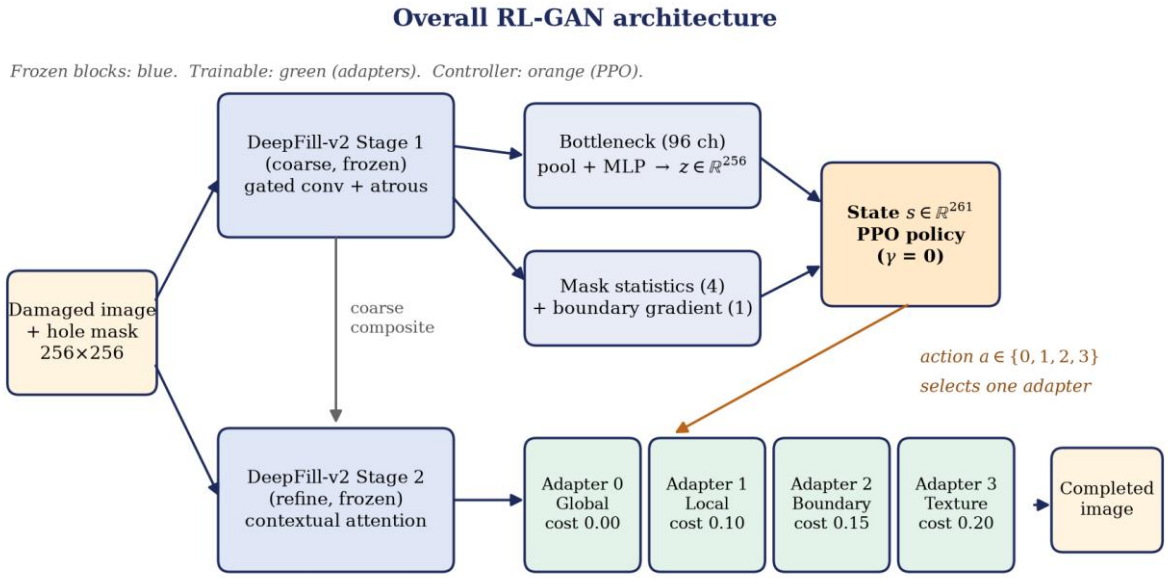


Figure 1: Overall RL-GAN architecture: frozen DeepFill-v2 backbone, 261-d state, PPO policy and four strategy adapters.

3.2 Frozen DeepFill-v2 backbone

The backbone is the PyTorch port of DeepFill-v2 by *nipponjo* [18] with Places2 weights (`deepfillv2_places2.pth`, about 40 MB), using *cnum* = 48 base channels and a five-channel input [masked RGB, ones, mask]. Stage 1 (coarse) uses gated convolutions with dilations up to 16, producing a coarse prediction. Stage 2 (refine) has a convolutional branch and a contextual-attention branch whose outputs are concatenated and decoded. All backbone parameters have `requires_grad = False` and the module is held in evaluation mode during adapter and PPO training, so every gain comes from the adapters and the controller.

3.3 State representation

The observation $s = [z, m, g] \in \mathbb{R}^{261}$ has three parts (Figure 2, left):

- **Latent embedding $z \in \mathbb{R}^{256}$.** The output of the last dilated layer of the coarse stage has $2 \cdot cnum = 96$ channels. It is globally average-pooled and passed through `Linear → LayerNorm → ELU → Linear` to give a 256-d vector. This costs no extra backbone pass beyond the coarse forward that is needed anyway.
- **Mask geometry $m \in \mathbb{R}^4$.** Missing-area ratio; bounding-box area ratio; number of connected components divided by 10 and clipped to [0, 1]; and centroid distance from the image centre normalised by the half-diagonal.
- **Boundary discontinuity $g \in \mathbb{R}^1$.** Mean Sobel gradient magnitude of the coarse composite on a one-pixel ring just outside the hole, divided by 2 and clipped to [0, 1].

$$g = \text{clip}((1/|B|) \Sigma_{p \in B} \|\nabla I_c(p)\|_2 / 2 , 0, 1) \quad (5)$$

3.4 Action space and strategy adapters

The agent has four discrete actions, each of which selects one adapter and has a hand-set cost penalty *C(a)* (Figure 2, right; Table 1). The costs encode a mild preference for the cheapest strategy when gains are equal; they are not measured compute, since all four adapters have nearly identical size and speed.

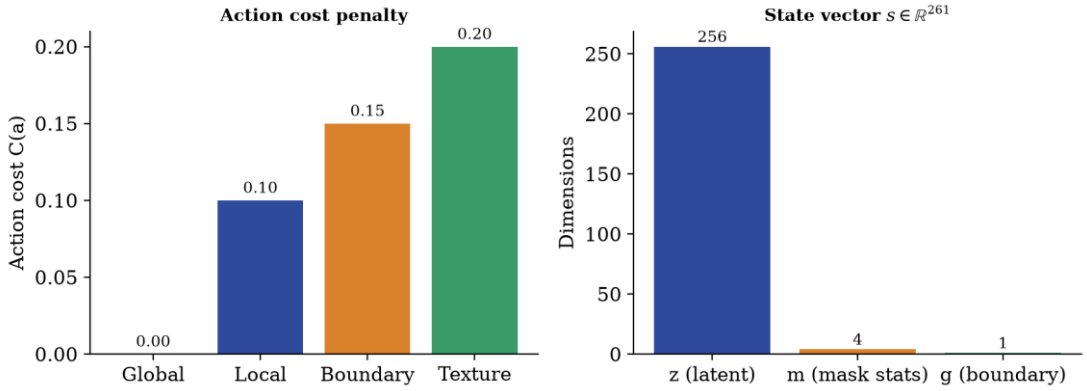


Figure 2: Left: action costs C(a). Right: composition of the 261-dimensional state vector.

Table 1: The four strategy adapters and their action costs.

Action	Adapter	Mechanism (from the implementation)	Cost
0	Global	Plain residual pass over the refined RGB image; no mask weighting.	0.00
1	Local	Input scaled by (1 + 0.5·M) so the hole is amplified before the residual block.	0.10
2	Boundary	Dilate-minus-erode ring (5×5) increases the blend weight near the hole edge.	0.15
3	Texture	Adds a Laplacian high-pass term inside the hole plus a small high-frequency branch.	0.20

All adapters share the same skeleton (Figure 3): `Conv(3→64) → InstanceNorm → ELU → FiLM → Conv(64→64) → InstanceNorm → ELU → Conv(64→3)`. They operate on the *RGB output* of the refine stage (not on internal features) and produce a bounded residual update:

$$\text{out} = \tanh(x + 0.5 \cdot \tanh(\Delta(x, e_a))) \quad (6)$$

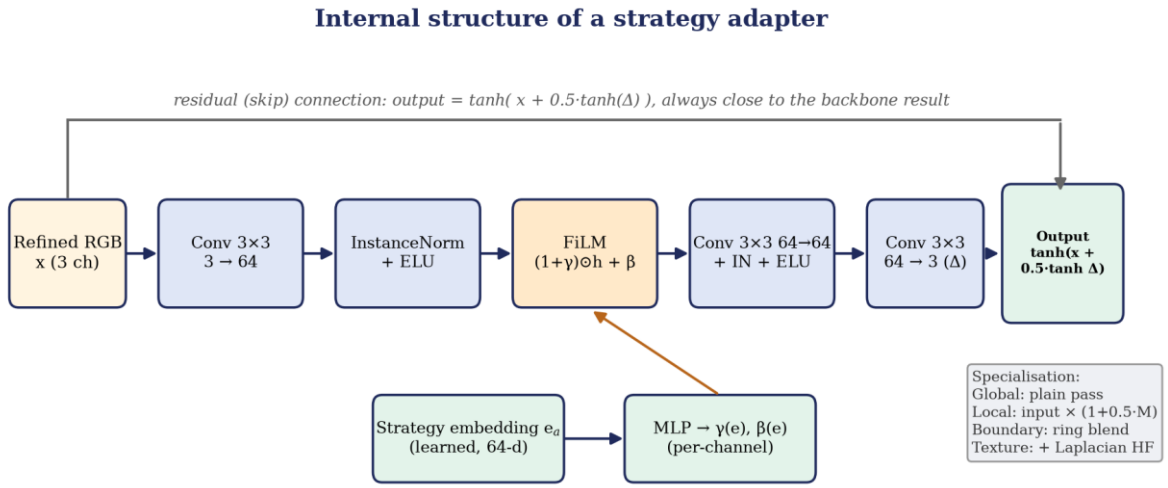


Figure 3: Internal structure of a strategy adapter with FiLM conditioning from the learned strategy embedding and a residual connection.

which keeps the output in [−1, 1] and close to the backbone result. The whole adapter set, including the four 64-d strategy embeddings, is only about 1.5 MB, against roughly 40 MB for the frozen backbone.

3.5 Adapter pretraining

The adapters are pretrained with the backbone frozen. For every batch, each adapter in turn receives the refined backbone output, is composited into the hole, and is trained with its own loss (Table 2). Optimisation uses Adam [17] with $lr = 10^{-4}$, $\beta = (0.5, 0.999)$, batch size 4, 256×256 images from COCO val2017 [15], dynamic random stroke masks, and 5 epochs, as stored in the checkpoint configuration.

Table 2: Strategy-specific pretraining objectives.

Adapter	Loss used in the code
Global (0)	L1 hole + L1 valid + VGG-16 [16] perceptual loss (weight 0.05)
Local (1)	6·L1 hole + 0.2·L1 valid + 0.5·SSIM-style term (L1 on 11×11 box-filtered means inside the hole)
Boundary (2)	L1 hole + L1 valid + 2·Sobel-gradient L1 weighted 2.0 on the hole-boundary ring and 0.2 elsewhere
Texture (3)	L1 hole + L1 valid + 0.1·L1 of FFT magnitudes (whole image) + VGG perceptual loss (weight 0.05)

3.6 Reward design

For one image and action a, the reward measures improvement of the refined result over the coarse composite, minus the action cost:

$$R = \alpha \cdot \Delta PSNR\tilde{} + \beta \cdot \Delta SSIM\tilde{} - \gamma \cdot \Delta LPIPS\tilde{} + \delta \cdot \Delta D - \lambda \cdot C(a) \quad (7)$$

The deltas are normalised and clipped to prevent PSNR from dominating: $\Delta PSNR\tilde{} = \text{clip}(\Delta PSNR / 10, \pm 2)$, $\Delta SSIM\tilde{} = \text{clip}(\Delta SSIM / 0.1, \pm 2)$, $\Delta LPIPS\tilde{} = \text{clip}(\Delta LPIPS / 0.1, \pm 2)$; ΔD

is the change in the mean patch confidence of a spectral-normalised PatchGAN discriminator, and the total is clipped to ±10 for stable PPO updates. The training configuration uses (α, β, γ, δ, λ) = (1.0, 1.0, 0.5, 0.3, 0.1).

3.7 PPO contextual-bandit environment

The environment (`InpaintingBanditEnv`) draws a random image and stroke mask from the dataset, runs the coarse stage, extracts the 261-d state, and exposes `Discrete(4)` actions (Figure 4). Each episode lasts one step: the chosen adapter is applied, the reward is computed from PSNR, SSIM, LPIPS and the discriminator, and a new sample is drawn. With γ = 0 the value function only has to predict the immediate reward.

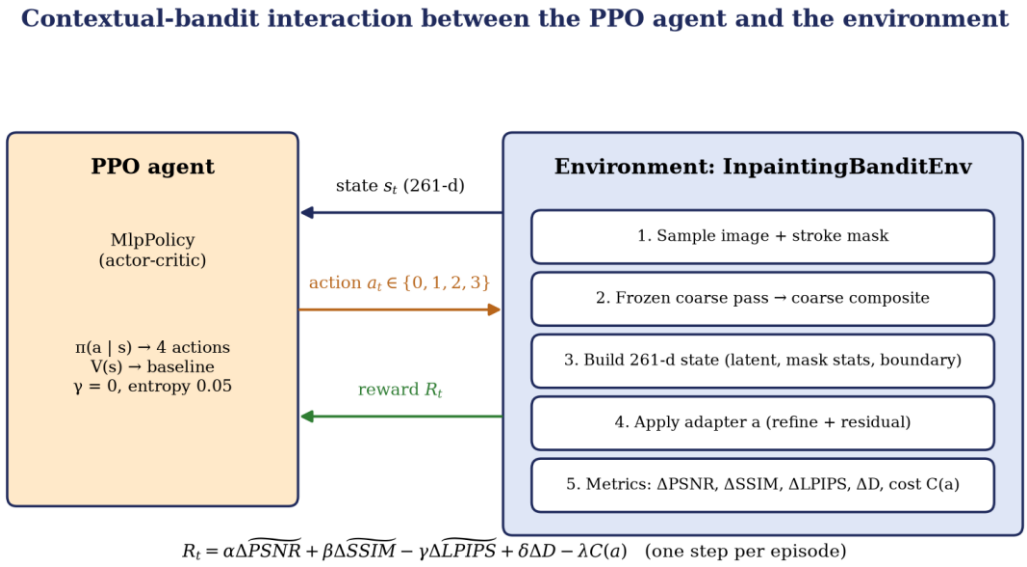


Figure 4: Agent–environment interaction in the one-step contextual bandit: state, action and reward.

Table 3: PPO configuration (configs/rl_agent_bandit.yaml).

Hyper-parameter	Value	Hyper-parameter	Value
Policy	MlpPolicy (actor-critic)	Learning rate	3×10^{-4}
Discount γ	0.0 (one-step bandit)	Rollout length n_steps	128
Entropy coefficient	0.05	Mini-batch / epochs	32 / 4
Value-loss coefficient	0.5	Max grad-norm	0.5
Reward weights	(1, 1, 0.5, 0.3, 0.1)	Seed	42
Backbone / adapters	Frozen during PPO	LPIPS in reward	Enabled (VGG)

3.8 Data, masks and severity buckets

Adapters and the controller are trained on COCO val2017 [19] (5,000 images, resized to 256×256) with randomly generated irregular stroke masks (1–4 strokes of random width and length, an occasional circle). For severity analysis we generate 100 fixed, reproducible masks, 25 per bucket, with a seeded generator (Figure 5): 10–20 %, 20–40 %, 40–60 % and 60 %+ missing area. A manifest pairs the first 100 images with the masks for repeatable evaluation.

Test scenes for the ablation are taken from the six sample photographs in `data/raw/sample_images`.

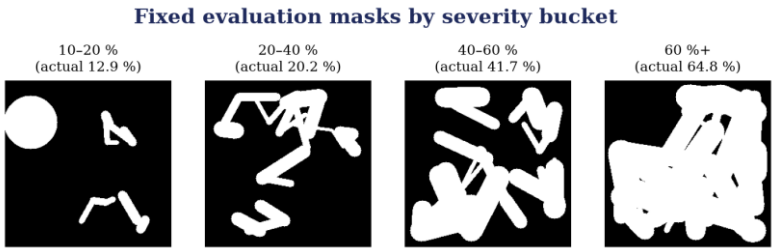


Figure 5: Examples of the fixed evaluation masks, one per severity bucket, with the measured missing-area percentage.

3.9 Evaluation protocol

Table 4 defines the four stages. Only stages A–C are valid generalisation measurements. Stage D optimises *all trainable parameters* (adapters, strategy embeddings and embedding head) for 400 AdamW steps (*lr* 1.5×10^{-3} with cosine decay to 2×10^{-5}) using a hole-weighted L1 loss and an 8× gradient-matching loss against the ground-truth image. It therefore shows capacity, not generalisation.

Table 4: The four-stage ablation protocol.

Stage	Configuration	Uses ground truth?	Valid test?
A	DeepFill-v2 backbone only (no adapter)	No	Yes
B	Fixed adapter 0 (Global)	No	Yes
C	PPO selects the adapter; single forward pass	No	**Yes: the RL measurement**
D	C + 400 AdamW steps on the test image	**Yes**	No (oracle)

4. IMPLEMENTATION

4.1 Software stack

Table 5: Software stack and hardware.

Component	Technology
Deep learning	PyTorch 2.6 (CUDA 12.4), torchvision (VGG-16), `lpips`
Reinforcement learning	Stable-Baselines3 (PPO), Gymnasium environments
Image / metrics	OpenCV (masks, Sobel), scikit-image (PSNR, SSIM), Pillow
Experiment management	YAML configs, TensorBoard logging, seeded runs (seed 42)
Web studio	Python `http.server` backend; HTML5 canvas, CSS and vanilla JavaScript front end
Hardware used	NVIDIA GeForce RTX 3050 GPU, Windows 11, Python 3.12

4.2 Code organisation

The repository is organised around the pipeline stages. `src/models` contains the backbone wrapper, the adapters and the unified `RLInpaintingModel`; `src/rl` contains the state builder, actions, reward and the Gymnasium environments; `src/training` contains adapter pretraining and PPO training; `src/evaluation` and `scripts/` contain the metrics, severity analysis and the ablation harness; `app.py` and `web/` implement the studio. A single `main.py` command-line entry point exposes twelve sub-commands (Table 6).

Table 6: Main commands of the pipeline.

Command	Purpose
create-eval-set	Generate the 100 fixed masks (25 per severity bucket) and manifest
pretrain-adapters	Pretrain the four adapters with the frozen backbone
verify-strategies	Check embedding variance and that adapters give different outputs, before PPO
train-rl	Train the PPO contextual-bandit controller
eval-baseline	Evaluate the backbone alone (Experiment A)
scripts/run_multi_image_test.py	Run the four-stage ablation and write the report and panels
train-gan, train-ablations, joint-finetune	Legacy / extension experiments built on an earlier small generator

4.3 Pipeline workflow

Figure 6 shows the order in which the stages are run; the verification gate must pass before PPO is trained.

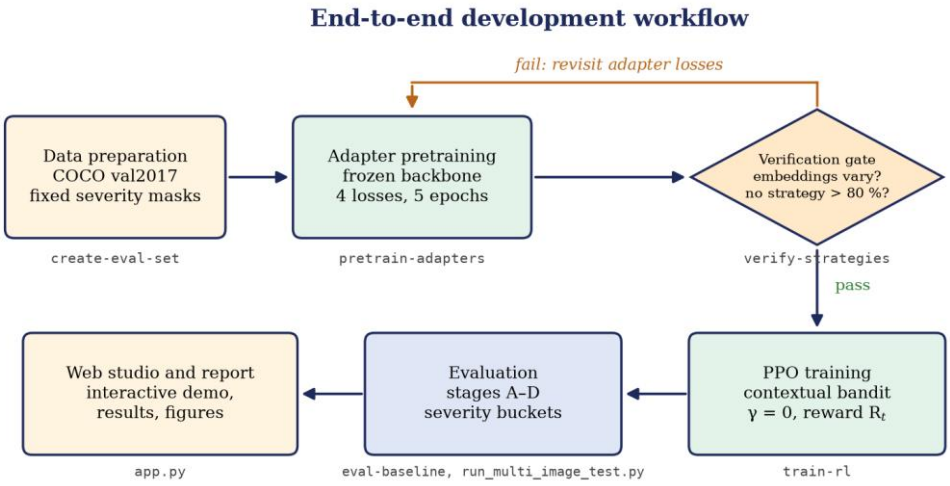


Figure 6: End-to-end development workflow with the commands that implement each stage.

1. Download the DeepFill-v2 Places2 weights and COCO val2017 images.
2. Generate the fixed evaluation masks (`create-eval-set`).
3. Pretrain the four adapters (`pretrain-adapters`) to obtain `adapters_final.pt`.
4. Run `verify-strategies` as a gate: state embeddings must vary across images and no single strategy may win more than 80 % of images.
5. Train PPO (`train-rl`) to obtain `ppo_bandit_final.zip`.
6. Evaluate with `eval-baseline` and `run_multi_image_test.py`, and demonstrate with `app.py`.

4.4 Interactive web studio

The studio (Figures 7 and 8) lets a user choose a preset photograph or upload one, paint an arbitrary mask with a variable brush, and run inpainting. The server builds the 261-d state, queries the PPO agent, applies the chosen adapter, and returns five images (ground truth, masked input, coarse result, backbone baseline, RL-GAN output) together with PSNR, SSIM, damaged-area percentage and latency. The front end provides a before/after split slider and a five-stage pipeline inspector. A second mode (*Ultra-Precision*) runs 60 AdamW steps on the chosen adapter against the original image; because the original image is the target, it shows how closely an adapter can fit a given image.

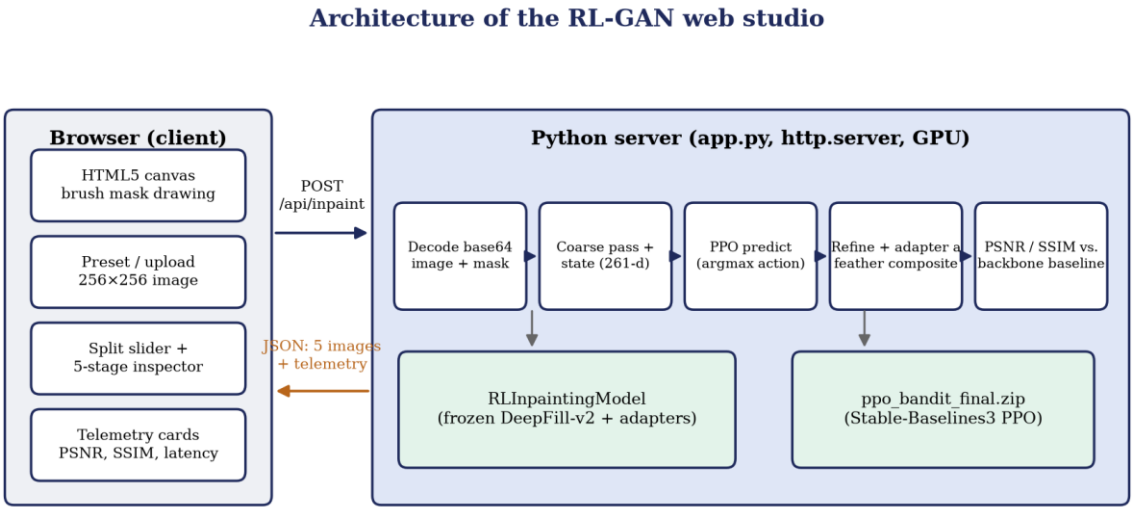


Figure 7: Client–server architecture of the web studio: canvas front end, JSON API and the GPU-resident model and PPO agent.

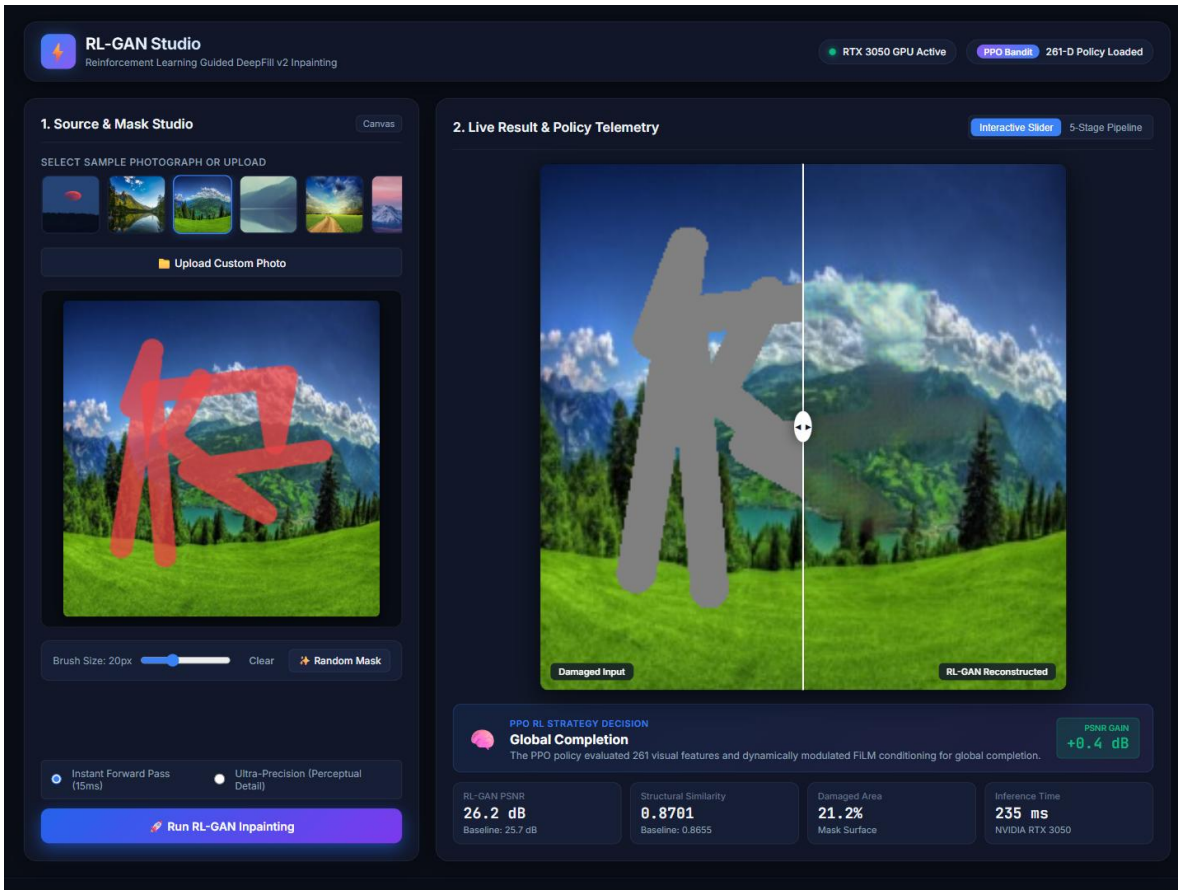


Figure 8: RL-GAN Studio running on the RTX 3050: the PPO agent selected Global Completion for the meadow preset (damaged area 21.2 %, 26.2 dB vs 25.7 dB baseline, 235 ms).

5. RESULTS AND ANALYSIS

5.1 Experimental setup

All numbers in this chapter were produced by the code in the repository on an NVIDIA RTX 3050 GPU. Images are 256×256 and PSNR/SSIM are computed on the composited full image in the [0, 1] range. The ablation uses three landscape photographs, each with a hand-designed mask made of two or three 15-pixel-wide line strokes and a small circular blob.

5.2 Backbone-only baseline (Experiment A)

Table 7: DeepFill-v2 backbone alone on five test samples (eval-baseline).

Metric	Mean	Std. dev.	n
PSNR (dB)	21.23	1.82	5
SSIM	0.864	0.029	5
LPIPS (VGG)	0.033	0.014	5
L1 (full image)	0.0566	0.0162	5
L1 (hole only)	0.4275	0.0745	5

The backbone alone reaches 21.2 dB on the five test samples. The L1 error inside the hole (0.43 on the [−1, 1] scale) is much higher than over the whole image (0.057), confirming that almost all of the error comes from the hole itself.

5.3 Do the four adapters behave differently?

Before training the controller we ran `verify-strategies` on five images with random masks: for each image the four adapters were applied to the same coarse composite (Table 8, Figures 9 and 10). The Local adapter gave the best PSNR on three of five images and the Global adapter on the other two; Boundary and Texture were never best. No strategy won more than 60 % of the images, so the 80 % gate was passed. Global and Local are within about 0.3 dB of each other on every image, and Boundary is at most 2.4 dB below the best adapter.

Table 8: PSNR (dB) of each adapter on the five verification images.

Image	Global	Local	Boundary	Texture	Best
0	30.28	30.18	28.97	29.82	Global
1	26.72	**27.04**	26.09	25.98	Local
2	25.94	**26.23**	24.16	24.90	Local
3	**23.12**	22.89	21.95	22.68	Global
4	30.35	**30.46**	28.04	28.42	Local
Mean	27.28	27.36	25.84	26.36	–

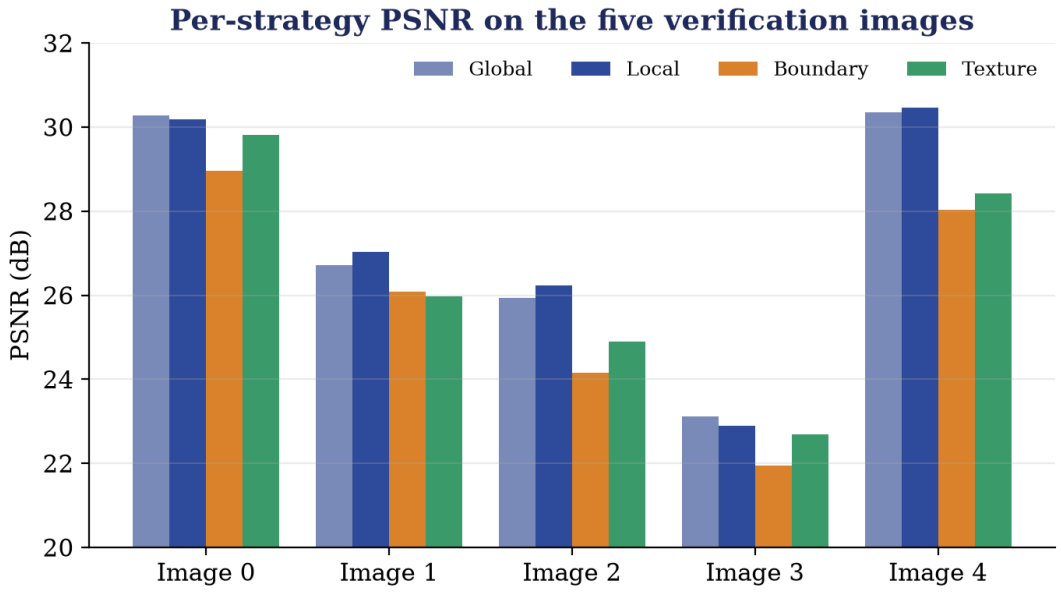


Figure 9: Per-strategy PSNR on the five verification images.



Figure 10: Verification image 2. Top row: ground truth, masked input, coarse composite, Global adapter. Bottom row: Global, Local, Boundary and Texture adapters.

5.4 Four-stage ablation

Table 9 and Figures 11–12 summarise the ablation on the three test scenes, and Figures 13–15 show the corresponding visual panels. PSNR differences are given relative to the backbone (Stage A).

Table 9: PSNR (dB) of the four stages. SSIM for stage C is 0.9730, 0.9734 and 0.9834 respectively.

Scene	PPO action	A: Backbone	B: Fixed Global	C: PPO-selected	D: Oracle
Alpine lake	2 (Boundary)	34.45	35.21	**32.25**	37.23
Lush meadow	0 (Global)	33.72	34.54	**34.54**	39.65
Sunset peaks	2 (Boundary)	40.58	40.00	**36.80**	43.19
Δ vs. A (C)		–		−2.21 / +0.82 / −3.78	

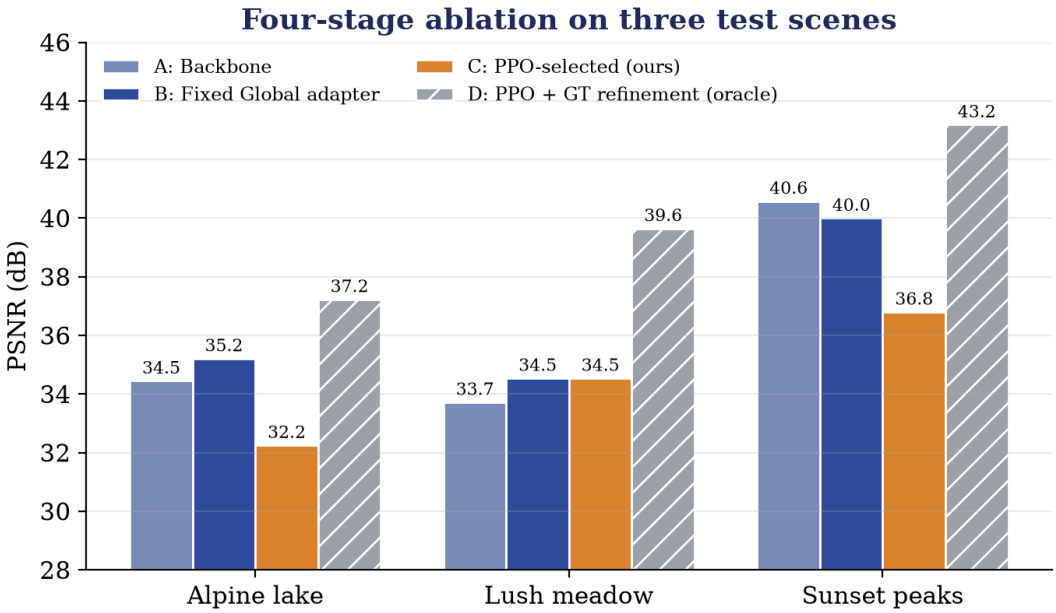


Figure 11: PSNR of the four ablation stages on the three test scenes.

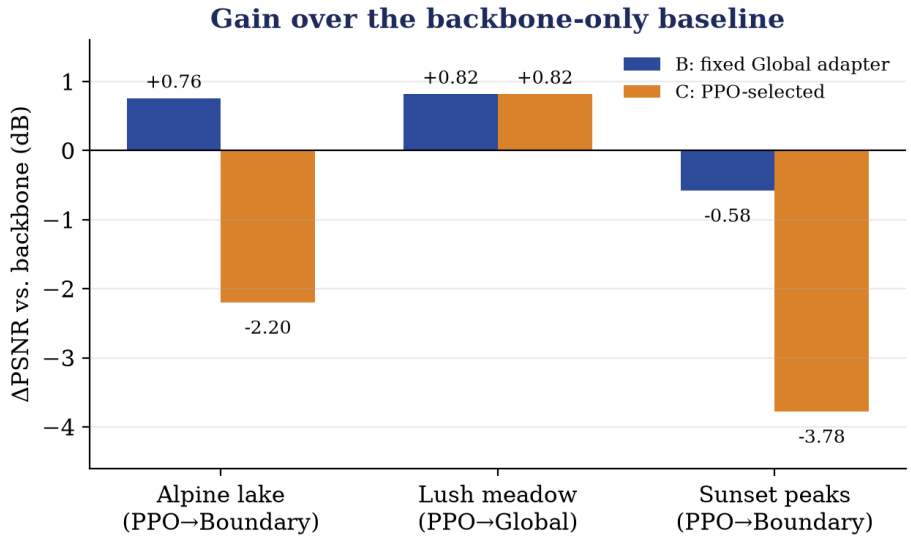


Figure 12: PSNR change relative to the backbone for the fixed Global adapter (B) and the PPO-selected adapter (C).

Reading the results. (i) The fixed Global adapter improves on the backbone in two of the three scenes (+0.76 dB lake, +0.82 dB meadow) and is worse on the sunset scene (−0.58 dB). (ii) On the meadow the PPO agent selected the Global adapter, so stage C equals stage B; the gain is therefore due to the adapter, not to adaptive selection. (iii) On the lake and sunset scenes PPO selected the Boundary adapter and the result is 2.2 dB and 3.8 dB *worse* than the backbone, and SSIM also drops slightly. (iv) The oracle (Stage D) reaches 37.2–43.2 dB, which shows that the adapter structure can represent a much better solution when it is given the ground truth for 400 steps.



Figure 13: Alpine lake. Top: ground truth, masked input, coarse result and backbone (A). Bottom: A, fixed adapter (B), PPO-selected Boundary adapter (C), oracle refinement (D).

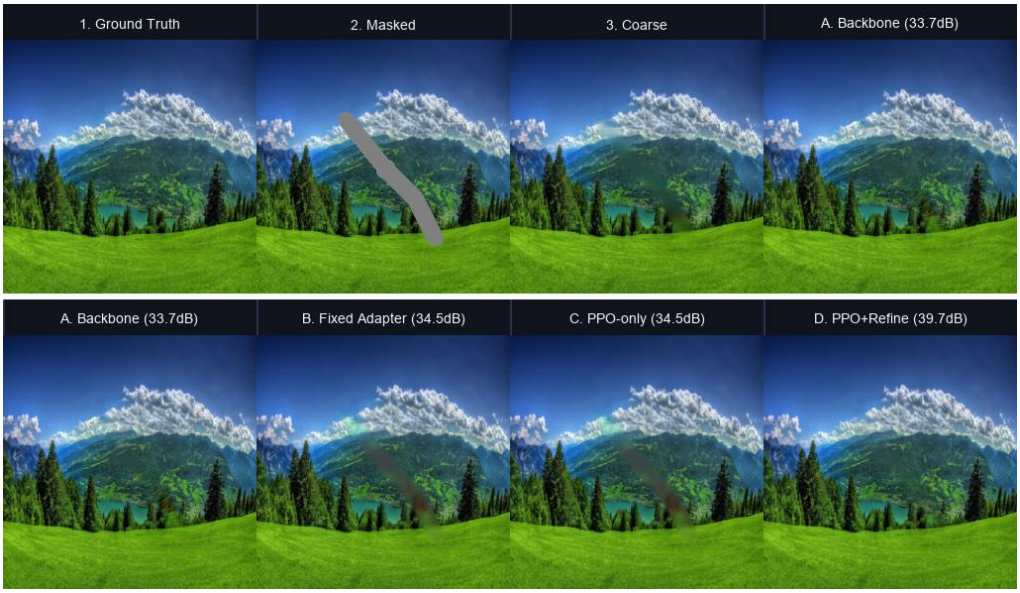


Figure 14: Lush meadow. PPO selects the Global adapter, so stages B and C coincide (34.5 dB).

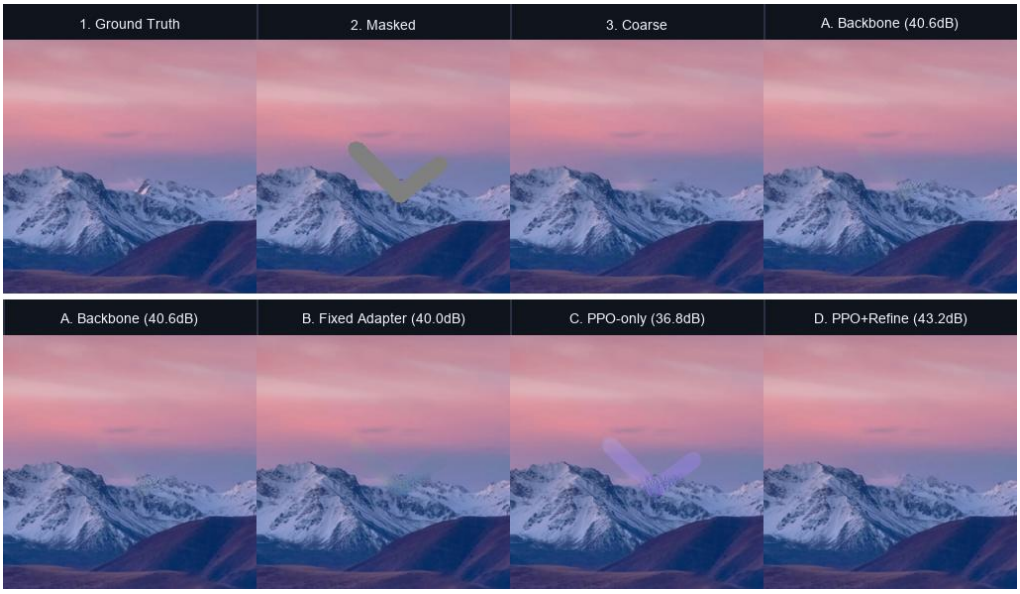


Figure 15: Sunset peaks. The Boundary adapter selected by PPO lowers PSNR to 36.8 dB relative to the 40.6 dB backbone.

5.5 Web-studio behaviour and latency

In the studio, the same code path is exercised on user-drawn masks. For the meadow preset in Figure 8 the agent selected the Global adapter and the telemetry card reported 26.2 dB against a 25.7 dB backbone baseline (+0.4 dB) with 235 ms latency. The latency figure covers the coarse pass, state extraction, the PPO decision, an additional backbone-only baseline pass, the adapter pass and CPU PSNR/SSIM computation; a pure adapter forward pass would be faster.

5.6 Discussion

The experiments support three conclusions. First, the pipeline works end to end: a frozen generator, specialised adapters, a state builder, a PPO bandit and a usable interface all run together. Second, small residual adapters give measurable gains (up to about +0.8 dB) when the Global adapter is used. Third, the learned controller selects different adapters for different scenes: Global on the meadow scene (where it matches the fixed Global adapter) and Boundary on the lake and sunset scenes. A direct baseline that always picks the adapter with the highest mean PSNR (Local in Table 8) is simple to evaluate and is a natural additional comparison. The severity question (Section 1.2) is addressed by the fixed masks and the severity evaluator; we describe the planned experiment in Chapter 6.

6. CONCLUSION AND FUTURE WORK

6.1 Conclusion

We designed, implemented and evaluated RL-GAN, a pipeline in which a PPO contextual-bandit controller selects among four specialised, FiLM-conditioned adapters placed on top of a frozen DeepFill-v2 inpainting generator. The state is obtained almost for free from the backbone bottleneck plus five scalar descriptors, the adapters are pretrained with Global, Local, Boundary and Texture objectives, and the system is exposed through a command-line pipeline and an interactive web studio.

The staged evaluation shows that the adapters give small improvements over the backbone when a fixed Global strategy is used (+0.76 dB and +0.82 dB on two of three scenes), that the adapter architecture can fit a specific image to 37–43 dB when given ground truth (the oracle), and that the learned controller chooses different strategies for different scenes: it selected the Global adapter on the meadow scene (matching the fixed Global adapter) and the Boundary adapter on the lake and sunset scenes.

The work therefore delivers a complete and honest experimental platform. Its main value is a clear, modular baseline against which better state encoders, rewards and policies can be compared.

6.2 Future work

- **Extend training.** Train PPO for more steps with logged learning curves, and try alternative reward weightings and state encoders.
- **Stronger baselines.** Add “always the best single adapter” and a supervised classifier trained on oracle best-adapter labels; compare PPO against both.
- **Test the severity hypothesis.** Evaluate on the 100 fixed masks across the four severity buckets with multiple seeds and confidence intervals, and report the action distribution per bucket.
- **Richer actions.** Allow multi-step refinement with a STOP action, and continuous mixing weights between adapters.
- **Larger and diverse data.** Train on a larger dataset than COCO val2017 and test on Places2 and CelebA-HQ.
- **Stronger references.** Compare with real pretrained LaMa and a diffusion inpainting model in terms of quality and latency.

REFERENCES

[1] J. Yu, Z. Lin, J. Yang, X. Shen, X. Lu and T. S. Huang, "Generative image inpainting with contextual attention," in *Proc. IEEE CVPR*, 2018, pp. 5505–5514.

[2] J. Yu, Z. Lin, J. Yang, X. Shen, X. Lu and T. S. Huang, "Free-form image inpainting with gated convolution," in *Proc. IEEE/CVF ICCV*, 2019, pp. 4471–4480.

[3] P. Isola, J.-Y. Zhu, T. Zhou and A. A. Efros, "Image-to-image translation with conditional adversarial networks," in *Proc. IEEE CVPR*, 2017, pp. 1125–1134.

[4] T. Miyato, T. Kataoka, M. Koyama and Y. Yoshida, "Spectral normalization for generative adversarial networks," in *Proc. ICLR*, 2018.

[5] R. Suvorov *et al.*, "Resolution-robust large mask inpainting with Fourier convolutions," in *Proc. IEEE/CVF WACV*, 2022, pp. 2149–2159.

[6] B. Zhou, A. Lapedriza, A. Khosla, A. Oliva and A. Torralba, "Places: A 10 million image database for scene recognition," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 40, no. 6, pp. 1452–1464, 2018.

[7] E. Perez, F. Strub, H. de Vries, V. Dumoulin and A. Courville, "FiLM: Visual reasoning with a general conditioning layer," in *Proc. AAAI*, 2018, pp. 3942–3951.

[8] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.

[9] L. Li, W. Chu, J. Langford and R. E. Schapire, "A contextual-bandit approach to personalized news article recommendation," in *Proc. WWW*, 2010, pp. 661–670.

[10] J. Schulman, F. Wolski, P. Dhariwal, A. Radford and O. Klimov, "Proximal policy optimization algorithms," *arXiv:1707.06347*, 2017.

[11] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus and N. Dormann, "Stable-Baselines3: Reliable reinforcement learning implementations," *J. Mach. Learn. Res.*, vol. 22, no. 268, pp. 1–8, 2021.

[12] M. Towers *et al.*, "Gymnasium: A standard interface for reinforcement learning environments," *arXiv:2407.17032*, 2024.

[13] Z. Wang, A. C. Bovik, H. R. Sheikh and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Trans. Image Process.*, vol. 13, no. 4, pp. 600–612, 2004.

[14] R. Zhang, P. Isola, A. A. Efros, E. Shechtman and O. Wang, "The unreasonable effectiveness of deep features as a perceptual metric," in *Proc. IEEE CVPR*, 2018, pp. 586–595.

[15] T.-Y. Lin *et al.*, "Microsoft COCO: Common objects in context," in *Proc. ECCV*, 2014, pp. 740–755.

[16] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *Proc. ICLR*, 2015.

[17] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. ICLR*, 2015.

[18] nipponjo, “deepfillv2-pytorch,” GitHub repository, https://github.com/nipponjo/deepfillv2-pytorch (Places2 DeepFill-v2 weights used as the frozen backbone).

[19] COCO Consortium, “COCO 2017 validation images (val2017),” http://images.cocodataset.org/zips/val2017.zip.

Project resources

Source repository: https://github.com/Hariprasaadh/rl-gan-inpainting. Datasets: COCO 2017 validation images (5,000 images) and six sample landscape photographs used for demonstrations.

APPENDIX A: REPRODUCTION COMMANDS

All commands are run from the repository root inside the project virtual environment (`.venv\Scripts\Activate.ps1` on Windows PowerShell).

```
python scripts/download_gan.py            # DeepFill-v2 Places2 weights
python scripts/download_coco.py           # COCO val2017 images
python main.py create-eval-set --image-dir data/raw/val2017 --output-dir
data/masks/fixed_eval_masks --count 25 --image-size 256
python main.py pretrain-adapters --config configs/pretrain_adapters.yaml
python main.py verify-strategies --data-dir data/raw/sample_images --num-images 5 --
output-dir results/strategy_verify
python main.py train-rl --config configs/rl_agent_bandit.yaml --timesteps 10000
python main.py eval-baseline --data-dir data/raw/sample_images --mask-dir
data/masks/fixed_eval_masks --samples 100
python scripts/run_multi_image_test.py    # four-stage ablation
python app.py                             # web studio at http://localhost:7860
```

APPENDIX B: ADDITIONAL QUALITATIVE RESULTS



Figure 16: Verification image 0 (same layout as Figure 10).



Figure 17: Verification image 4 (same layout as Figure 10).